Scanner

Scanner - t0xodile, GitHub.

- Published: date not stated
- Original: <https://github.com/t0xodile/crlf-powered-desync-scanner>
- Preserved from: https://github.com/t0xodile/crlf-powered-desync-scanner (git) on 2026-08-08
- Repository commit: 8b21e786dfe46784bad389a9ebf8dd0f877eabb4
- Licence: see the repository

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/t0xodile/crlf-powered-desync-scanner>
- Commit: `8b21e786dfe46784bad389a9ebf8dd0f877eabb4`
- Documents preserved: 1

README.md

Blob `b3223fa16847`, 1699 bytes, at commit `8b21e786dfe4`.

CRLF-Powered Desync Scanner

This is the Burp Extension we used to detect all cases from our research CRLF-Powered Desync Attacks: Beheading HTTP Streams.

It's a research-first scanner built using BulkScan similar to HTTP Request Smuggler and others.

Implementation

Request Header Injection probe

Uses all the defined mutations in `src/main/kotlin/ReqMutator` to fire a benign and then probe request. If the response contains an expected match, or differs significantly, an issue is reported.

If the response contained **all** expected matches, then the issue is reported as normal. If the fallback diff is enabled (uses `serverStatus` from `BulkScan`) then the issue is reported as `Dodgy`.

There is also a `WAFChecker` which uses static strings to try and reduce FPs from WAF rules and follow-up logic that ensures the behaviour is **consistent** or at least most likely not a WAF rule that isn't in the `WAFChecker` list already. Additionally, there is an "auto-exploit via Response Queue Poisoning" button that may or may not give you a clue that RQP works out of the box.

Response Header Injection probe

Injects a header containing a random canary into the request path and checks whether that header is reflected back in the response headers. If it is, the injection is confirmed and an issue is reported.

As a follow-up, it then injects a `Content-Length` header to see if the response can be split. If this causes a timeout or a change in the `serverStatus`, the issue is upgraded and reported as `Splitting?`.
